

Analyse Technique

Connexion Supabase & QR Code Smartphone

VTTApp - Application de Table de Jeu Virtuelle

Document généré le 22 mai 2026

Version de l'application : v0.709

Département Technique - Analyse d'Architecture

Table des Matières

1. Architecture Globale de la Connexion

- 1.1 Vue d'ensemble du système
- 1.2 Schéma de communication

2. Connexion Supabase

- 2.1 Initialisation du client
- 2.2 Cascade de résolution des credentials
- 2.3 Bootstrap au démarrage
- 2.4 Realtime Broadcast (couche de communication)
- 2.5 Stockage de fichiers (Storage)
- 2.6 Interface utilisateur de configuration

3. Génération des QR Codes

- 3.1 Bibliothèque utilisée
- 3.2 QR Codes disponibles
- 3.3 Construction des URLs

4. Flux de Connexion Smartphone

- 4.1 Étape 1 : Création de la salle
- 4.2 Étape 2 : Partage via QR Code
- 4.3 Étape 3 : Scan et rejoindre
- 4.4 Étape 4 : Connexion Realtime
- 4.5 Étape 5 : Traitement de la demande
- 4.6 Étape 6 : Interface Smartphone

5. Interactions Smartphone (Tags)

- 5.1 Configuration des tags smartphone
- 5.2 Sélection de joueurs
- 5.3 Actions déclenchées

6. Sécurité et Gestion des Erreurs

7. Dépendances et Configuration

1. Architecture Globale de la Connexion

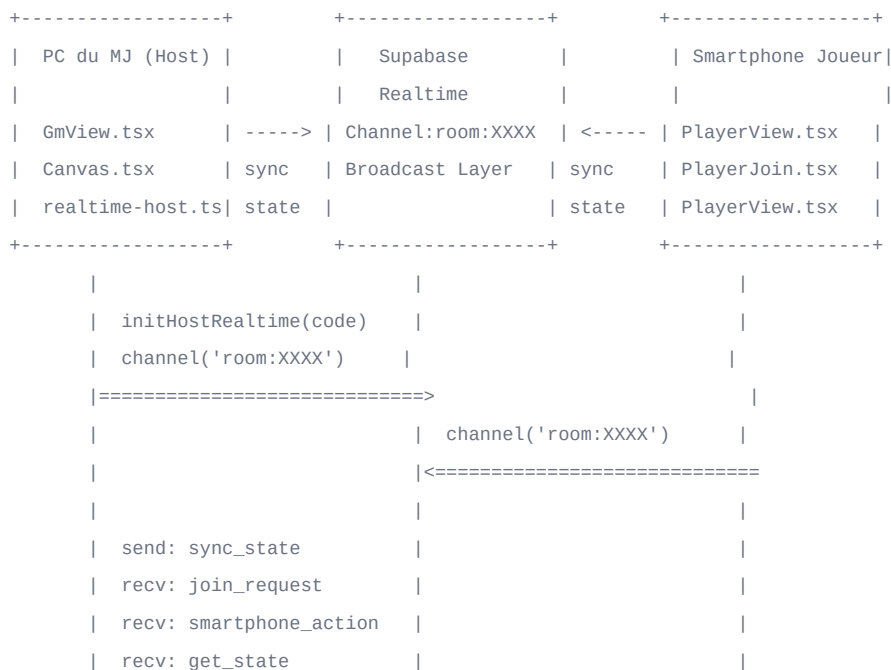
— 1.1 Vue d'ensemble du système

L'application VTApp repose sur une architecture client-serveur où Supabase agit comme infrastructure de communication en temps réel (Realtime Broadcast). Le MJ (Game Master) héberge la partie sur son navigateur et les joueurs se connectent via leurs smartphones en scannant un QR Code.

Points clés de l'architecture :

- Pas de serveur dédié : la logique métier s'exécute entièrement dans le navigateur du MJ
- Supabase Realtime sert de bus de communication entre le MJ et les smartphones
- Les joueurs n'ont pas besoin de compte : ils rejoignent via un code de salle et leur pseudo
- Le State (Zustand) du MJ est la source de vérité, diffusée en temps réel aux joueurs

— 1.2 Schéma de communication



2. Connexion Supabase

— 2.1 Initialisation du client

Fichier : src/lib/supabase.ts

Le client Supabase est initialisé via `createClient()` de `@supabase/supabase-js`. Si les credentials sont invalides ou absents, le client est null et les fonctionnalités nécessitant Supabase sont désactivées.

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = getEnvUrl() as string;
const supabaseAnonKey = getEnvKey() as string;

let sbClient = null;
try {
  if (supabaseUrl && supabaseAnonKey && supabaseUrl.startsWith('http')) {
    sbClient = createClient(supabaseUrl, supabaseAnonKey);
  }
} catch (error) {
  console.error('Invalid Supabase configuration', error);
}

export const supabase = sbClient; // Peut être null
```

— 2.2 Cascade de résolution des credentials

L'URL et la clé anon sont résolues selon une priorité stricte :

Pour l'URL Supabase (`getEnvUrl`) :

- 1. Paramètre d'URL ?surl=... (base64, stocké en sessionStorage)
- 2. sessionStorage 'VTT_SB_URL_OVERRIDE'
- 3. localStorage 'VTT_SUPABASE_URL'
- 4. import.meta.env.VITE_SUPABASE_URL (fichier .env)
- 5. import.meta.env.NEXT_PUBLIC_SUPABASE_URL

Pour la clé anon (`getEnvKey`) :

- 1. Paramètre d'URL ?sbkey=... (base64, stocké en sessionStorage)
- 2. sessionStorage 'VTT_SB_KEY_OVERRIDE'
- 3. localStorage 'VTT_SUPABASE_ANON_KEY'
- 4. import.meta.env.VITE_SUPABASE_ANON_KEY (fichier .env)
- 5. import.meta.env.NEXT_PUBLIC_SUPABASE_ANON_KEY

```
export const getEnvUrl = () => {
  const params = new URLSearchParams(window.location.search);
  let url = params.get('sburl');
  if (url) {
    try { url = atob(decodeURIComponent(url)); } catch(e) {}
    sessionStorage.setItem('VTT_SB_URL_OVERRIDE', url);
    return url;
  }
  const sessionUrl = sessionStorage.getItem('VTT_SB_URL_OVERRIDE');
  if (sessionUrl) return sessionUrl;
  return localStorage.getItem('VTT_SUPABASE_URL') ||
    import.meta.env.VITE_SUPABASE_URL ||
    import.meta.env.NEXT_PUBLIC_SUPABASE_URL;
};
```

— 2.3 Bootstrap au démarrage

Fichier : src/main.tsx

Au chargement de l'application, le bootstrap vérifie les paramètres d'URL pour injecter les credentials Supabase :

```
// Lecture des paramètres d'URL
const sburl = searchParams.get('sburl');
const sbkey = searchParams.get('sbkey');

if (sburl && sbkey) {
  localStorage.setItem('VTT_SUPABASE_URL', atob(sburl));
  localStorage.setItem('VTT_SUPABASE_ANON_KEY', atob(sbkey));
  sessionStorage.setItem('VTT_SB_URL_OVERRIDE', atob(sburl));
  sessionStorage.setItem('VTT_SB_KEY_OVERRIDE', atob(sbkey));
  // Redirection propre en conservant ?code=...
  window.location.replace(newUrl);
}
```

- Permet aux QR codes d'encoder les credentials pour les nouveaux utilisateurs
- Une redirection nettoie l'URL des paramètres sburl/sbkey
- Fallback : lecture du fichier /env.example si VTT_USE_ENV_EXAMPLE est actif

— 2.4 Realtime Broadcast (couche de communication)

Fichier : src/lib/realtime-host.ts

C'est le cœur de la communication en temps réel. Le MJ ouvre un canal Supabase Realtime avec broadcast et presence.

```
currentChannel = supabase.channel(`room:${roomCode}`, {
  config: {
    broadcast: { self: true, ack: false },
    presence: { key: 'host' }
  },
});
```

Événements gérés par le canal :

- join_request : un joueur demande à rejoindre
- get_state : un joueur demande l'état complet
- smartphone_action : action d'un joueur depuis son téléphone
- poll_response : réponse à un sondage
- group_vote_response : vote de groupe
- soundboard_action / soundboard_stop : contrôle de la soundboard à distance
- checklist_action : interaction avec la checklist
- action_trigger : déclenchement d'action

Diffusion de l'état (sync_state)

L'état complet est diffusé avec un système de retry pour assurer la livraison :

```
const doSend = (attempt) => {
  currentChannel.send({
    type: 'broadcast',
    event: 'sync_state',
    payload: buildFullStatePayload()
  }).then(() => console.log(`OK (attempt ${attempt+1})`))
  .catch(err => console.error(`Failed (attempt ${attempt+1})`, err));
};
doSend(0); // Envoi immédiat
setTimeout(() => doSend(1), 500); // +500ms
setTimeout(() => doSend(2), 1200); // +1.2s
setTimeout(() => doSend(3), 3000); // +3s (sécurité)
```

Le broadcast est débouncé à 150ms pour éviter les envois trop fréquents, et la position (pan/zoom) est throttlée à 500ms.

Note : Un système de diff a été implémenté puis abandonné car le client smartphone n'écoute que l'événement 'sync_state'. L'état complet est toujours envoyé.

— 2.5 Stockage de fichiers (Storage)

Supabase Storage est utilisé avec le bucket 'images-all' pour :

- Upload d'images de joueurs, rôles, tags, handouts
- Génération de noms de fichier uniques (random + timestamp)
- Suppression de fichiers via leur URL publique

```
export const uploadFileToStorage = async (file: File) => {
  const fileName = `${Math.random().toString(36).substring(2,15)}_${Date.now()}.${fileExt}`;
  const { error: uploadError } = await supabase.storage
    .from('images-all').upload(filePath, file, {
      cacheControl: '3600', upsert: false
    });
  if (uploadError) return null;
  const { data } = supabase.storage.from('images-all').getPublicUrl(filePath);
  return data.publicUrl;
};
```

— 2.6 Interface utilisateur de configuration

Fichier : src/components/layout/RightPanel.tsx

Un panneau de configuration permet au MJ de modifier les credentials à l'exécution :

- Champs URL / Clé Anon avec références vers les inputs
- Sauvegarde dans localStorage + rechargement de la page
- Option 'Utiliser .env.example' pour charger depuis le fichier serveur
- Bouton de validation avec appel à window.location.reload()

```
const saveSupabaseConfig = () => {
  localStorage.setItem('VTT_SUPABASE_URL', urlRef.current.value);
  localStorage.setItem('VTT_SUPABASE_ANON_KEY', keyRef.current.value);
  window.location.reload();
};
```

3. Génération des QR Codes

— 3.1 Bibliothèque utilisée

La bibliothèque 'qrcode.react' (version ^4.2.0) est utilisée pour générer les QR codes. Elle produit des SVG que l'utilisateur peut scanner avec n'importe quelle application de lecture de QR Code.

```
import { QRCodeSVG } from 'qrcode.react';  
  
<QRCodeSVG value={url} size={150} />
```

— 3.2 QR Codes disponibles

Fichier : src/components/layout/Canvas.tsx (lignes 855-965)

Un popup de connexion affiche deux QR Codes distincts :

QR Code 1 : Connexion Joueurs (Smartphone)

- URL générée : /join?surl=...&sbkey=...&code=ROOMCODE
- Icône : Smartphone (lucide-react)
- Couleur : bleue
- Permet aux joueurs de rejoindre la partie en scannant
- Taille du QR Code : 150x150px

QR Code 2 : Télécommande MJ (Soundboard)

- URL générée : /remote?surl=...&sbkey=...&code=ROOMCODE
- Icône : Radio (lucide-react)
- Couleur : rose
- Permet de contrôler la soundboard à distance
- Nécessite le code de configuration (passcode)

— 3.3 Construction des URLs

Les URLs sont construites dynamiquement en fonction des credentials disponibles :


```

let sbParams = '';
const sbUrl = getEnvUrl();
const sbKey = getEnvKey();

// N'ajoute les params que si on utilise localStorage (pas Vite env vars)
if (!import.meta.env.VITE_SUPABASE_URL && sbUrl && sbKey) {
  sbParams = `?sburl=${encodeURIComponent(btoa(sbUrl))}`
    + `&sbkey=${encodeURIComponent(btoa(sbKey))}`;
}

// Ajout du code de salle si activé
if (displaySettings.includeRoomCodeInLinks && roomCode) {
  const codeParam = `code=${encodeURIComponent(roomCode)}`;
  joinParams += (joinParams ? '&' : '?') + codeParam;
}

const joinHref = `${window.location.origin}/join${joinParams}`;
const sbHref = `${window.location.origin}/remote${remoteParams}`;

```

Les credentials sont encodés en base64 (via btoa) pour être passés en paramètres d'URL. Si l'application utilise les variables d'environnement Vite, les credentials ne sont pas inclus dans l'URL.

4. Flux de Connexion Smartphone

— 4.1 Étape 1 : Création de la salle

Le MJ génère un code de salle unique (6 lettres majuscules) et initialise le canal Realtime :

```
// Génération du code (dans le store Zustand)
generateRoomCode() -> 'ABCDEF'

// Initialisation du canal host
initHostRealtime('ABCDEF')
// -> ouvre supabase.channel('room:ABCDEF')
```

— 4.2 Étape 2 : Partage via QR Code

Le MJ clique sur le bouton QR Code dans la barre d'outils du Canvas. Un popup s'affiche avec les deux QR codes.

Le joueur scanne le QR Code avec son smartphone. L'URL encodée contient :

- Les credentials Supabase (URL + clé anon) en base64
- Le code de la salle
- Le chemin /join pour la page de connexion

— 4.3 Étape 3 : Scan et rejoindre

Fichier : src/pages/PlayerJoin.tsx

Le joueur arrive sur la page de connexion avec le code de salle pré-rempli :

```
// À l'arrivée : récupération du code depuis l'URL
const urlCode = searchParams.get('code')?.toUpperCase() || '';
sessionStorage.setItem('VTT_JOIN_ROOM_CODE', urlCode);

// Le joueur entre son pseudo et clique sur 'Rejoindre'
const handleJoin = (e) => {
  e.preventDefault();
  navigate(`/player/${roomCode}/${encodeURIComponent(playerName)}`);
};
```

— 4.4 Étape 4 : Connexion Realtime

Fichier : src/pages/PlayerView.tsx

Le composant PlayerView se monte avec le roomId et le playerName dans l'URL. Il crée son propre canal Realtime :

```
const channel = supabase.channel(`room:${roomId}`, {
  config: { broadcast: { ack: false }, presence: { key: playerName } },
});

channel
  .on('broadcast', { event: 'feedback_popup' }, ...)
  .on('broadcast', { event: 'sync_state' }, async ({ payload }) => {
    // Traitement de l'état reçu
    setRoomPlayers(data.players || []);
    setAllRoles(data.roles || []);
    // ...
  })
  .subscribe();
```

Le joueur envoie deux messages immédiatement après connexion :

- join_request { playerName } -> annonce sa présence
- get_state -> demande l'état actuel du jeu

Un mécanisme de retry toutes les 2 secondes relance la demande si non reconnu.

— 4.5 Étape 5 : Traitement de la demande

Fichier : src/lib/realtime-host.ts

Le MJ reçoit la demande et réagit selon le mode de la salle :

```
currentChannel.on('broadcast', { event: 'join_request' }, ({ payload }) => {
  const existingPlayer = getPlayerByName(players, payload.playerName);

  if (!existingPlayer) {
    if (state.isRoomPublic) {
      // Mode public : ajout automatique du joueur
      state.addPlayer({ name, color, x, y, ... });
      sendFullStateWithRetry();
    } else {
      // Mode privé : mise en file d'attente
      state.addJoinRequest(playerName);
      sendFullStateWithRetry();
    }
  } else {
    // Joueur existant : renvoi de l'état
    sendFullStateWithRetry();
  }
});
```

Mode public : le joueur est automatiquement ajouté au plateau de jeu.

Mode privé : le MJ doit approuver chaque demande via une liste d'attente.

— 4.6 Étape 6 : Interface Smartphone

Une fois connecté, le joueur voit l'interface smartphone avec les onglets configurables par le MJ :

Onglets disponibles :

- Game (jeu) : tags interactifs, actions, chronomètre
- Players : liste des joueurs avec leurs infos
- Room : fiche de la salle (background, description)
- Wiki : documentation de la partie
- Handouts : documents partagés par le MJ
- Logs : historique des actions

Fonctionnalités de l'interface :

- Affichage en temps réel des tags avec `showOnSmartphone: true`
- Sélecteur de joueurs (simple ou multiple) avec filtres (vivant/mort/rôle/équipe/soi-même)
- Boutons d'action avec feedback visuel
- Popup de révélation de rôle avec animation
- Notifications de jets de dés
- Effets de particules (confettis, sang, magie, feu, poison)
- Compteur de temps partagé
- Notes privées sur les joueurs (localStorage)
- Vibration du téléphone (via l'API Vibration)
- Sondages et votes de groupe

Le MJ peut contrôler à distance l'interface du joueur :

- Verrouiller l'écran (`isSmartphoneLocked`)
- Forcer un onglet spécifique (`forceSmartphoneTab`)
- Déclencher des popups d'information
- Faire vibrer le téléphone

— 5.1 Configuration des tags smartphone

Fichier : `src/components/tags/TagSmartphoneTab.tsx`

Le MJ configure les interactions smartphone via un onglet dédié dans l'édition des tags :

```
const RETURN_INFO_OPTIONS = [
  { key: 'none', label: 'Aucun' },
  { key: 'real_role', label: 'Rôle réel' },
  { key: 'real_team', label: 'Équipe réelle' },
  { key: 'seen_role', label: 'Vu comme rôle' },
  { key: 'seen_team', label: 'Vu dans l'équipe' },
];
```

Options de configuration disponibles :

- Mode de sélection : aucun / simple / multiple
- Filtres de sélection : vivant, mort, rôle, équipe, soi-même, pas soi-même
- Texte du bouton et message de feedback
- Vérification de rôle avec comptage ou réponse vague
- Retour d'information (rôle réel/vu, équipe réelle/vue)
- Fusion de tag (auto-assigner un tag aux joueurs sélectionnés)
- Auto-suppression du tag après utilisation
- Déclenchement d'action du moteur de jeu
- Association à un handout

— 5.2 Sélection de joueurs

Le joueur peut sélectionner des cibles sur son smartphone :

```
const togglePlayerSelection = (tagInstanceId, targetPlayerId) => {
  setSelectedPlayersByTag(prev => {
    const current = prev[tagInstanceId] || [];
    if (current.includes(targetPlayerId)) {
      return { ...prev, [tagInstanceId]: current.filter(id => id !== targetPlayerId) };
    } else {
      return { ...prev, [tagInstanceId]: [...current, targetPlayerId] };
    }
  });
};
```

— 5.3 Actions déclenchées

Fichier : src/lib/realtime-host.ts

Le MJ traite les actions smartphone et peut effectuer :

- Affichage de pastilles de sélection sur les joueurs ciblés
- Révélation d'info (rôle/équipe) via feedback_popup
- Vérification de rôle avec réponse personnalisée
- Fusion de tags sur les joueurs sélectionnés ou sur le joueur source
- Déclenchement d'actions du moteur de jeu avec contexte (\$Joueur, \$Cible)
- Suppression automatique du tag de l'interface smartphone

```
// Exemple : traitement du retour d'information
if (payload.smartphoneReturnInfo === 'real_role') {
  const role = state.roles.find(r => r.id === target.roleId);
  val = role?.name || 'Sans Rôle';
}

// Envoi du message au joueur
currentChannel.send({
  type: 'broadcast', event: 'feedback_popup',
  payload: { playerId: payload.playerId, message: infoMsg }
});
```

6. Sécurité et Gestion des Erreurs

— 6.1 Sécurité

- Les credentials Supabase sont stockés côté client (localStorage/sessionStorage)
- Les clés sont des clés anon (publiques, sécurisées par les RLS policies côté Supabase)
- Le passcode de la soundboard distante est vérifié côté MJ avant exécution
- Les credentials sont encodés en base64 dans les URLs (non sécurisé mais obfusqué)
- Aucune authentification utilisateur n'est requise (pas de compte)

— 6.2 Gestion des erreurs

- Client Supabase null si credentials invalides -> graceful degradation
- Retry sur les broadcasts (0, 500ms, 1.2s, 3s) pour fiabilité
- Timeout de connexion : retry toutes les 2s pour les joueurs
- Protection contre les payloads > 200KB (avertissement console)
- Strip des images data: URI trop grandes dans les broadcasts
- Préservation du paramètre ?code= lors du nettoyage des credentials d'URL

— 6.3 Limites et considérations

- Pas de serveur relais : le navigateur du MJ est le point central
- Si le MJ ferme son navigateur, tous les joueurs sont déconnectés
- Les images volumineuses sont retirées du broadcast pour éviter les problèmes de taille
- Le système de diff a été abandonné : l'état complet est toujours envoyé

7. Dépendances et Configuration

— Package.json

```
Dépendances Supabase :
"@supabase/supabase-js": "^2.99.3"

Dépendances QR Code :
"qrcode.react": "^4.2.0"

Dépendances associées :
"zustand": "^5.0.11"      (gestion d'état)
"react-router-dom": "^7.13.1" (routage)
"lucide-react": "^0.577.0"  (icônes)
"uuid": "^13.0.0"          (génération d'IDs)
```

— Fichier .env

```
VITE_SUPABASE_URL=https://qxtknkhjvhovradjwmfh.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_lyk52r8Cy5Y_F8K140rCRw_51yurkaD
```

— Architecture des fichiers clés

```
src/  
  lib/  
    supabase.ts      # Client Supabase + Storage  
    realtime-host.ts  # Canal Realtime côté MJ  
  pages/  
    PlayerJoin.tsx    # Page de connexion smartphone  
    PlayerView.tsx    # Interface smartphone complète  
  components/  
    layout/  
      Canvas.tsx      # QR Code popup (ligne 855)  
      RightPanel.tsx  # Configuration Supabase  
    tags/  
      TagSmartphoneTab.tsx # Config des tags smartphone  
  main.tsx            # Bootstrap credentials  
  store/  
    index.ts          # Zustand store (état global)
```